Dead on Arrival: Characterizing and Protecting Against Dead-Entry TLB Misses in GPU Microarchitectures

Abstract

GPU workloads with large memory footprints frequently suffer from redundant L2 TLB misses in which a recently evicted translation is immediately re-walked at full page-walk cost. We characterize these dead-entry misses *across 24 GPU workloads, finding they account for up to 99% of L2 TLB misses in the most TLB-sensitive applications, yet their performance impact varies widely depending on memory access structure. Workloads where warps share the same virtual page suffer from burst amplification, where a single eviction stalls many warps simultaneously waiting for one translation to return. In contrast, workloads where each warp accesses a distinct set of pages face a capacity-overflow problem that no replacement policy can resolve, a distinction validated by huge page experiments. Building on this two-class taxonomy, we design* DEPOT *(Dead-Entry PrOTection), a 1 KB Bloom filter mechanism that prevents recently evicted translations from being displaced immediately upon reinstallation, delivering up to 72% IPC improvement on interference-driven workloads with zero overhead on others, and composing with the state-of-the-art TLB prefetching and compaction mechanism, for 2 to 7% additional gain.*

***Index Terms*—GPU address translation, TLB, MSHR, memory hierarchy**

1. Introduction

Address translation is a first-order performance concern in modern GPU workloads [57]. As applications move toward larger, irregular memory footprints such as sparse graph traversal, scientific stencils, and machine learning inference over large embedding tables, the GPU L2 TLB becomes a sustained bottleneck [23, 61]. Prior work has approached this problem almost exclusively from an access-pattern perspective, predicting which virtual pages will be needed next and prefetching their translations before the demand miss occurs [4, 10, 28, 48]. This framing implicitly treats each TLB miss as a cold event requiring a fresh page walk, but many TLB misses are not cold at all. A *dead-entry TLB miss* occurs when a virtual page whose translation is still valid is evicted from the TLB and then re-accessed before the working set rotates again. The page walk is not cold; it is redundant. The entry was alive, was discarded by LRU replacement, and is now being reconstructed at full page-walk cost. Measuring this phenomenon across 24 GPU workloads reveals that dead-entry misses are pervasive: in the most TLB-sensitive applications, over 99% of all L2 TLB misses carry an eviction-history signature.

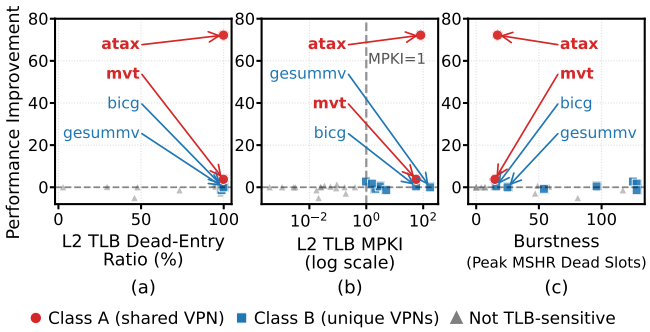

Figure 1: Performance improvement (IPC) from eliminating dead-entry re-walks against three candidate predictors: (a) DE ratio, (b) MPKI, and (c) MSHR burstiness, for all 24 workloads.

Yet the performance consequences of dead-entry misses are not uniform, and Figure 1 shows why simple miss statistics fail to explain the divergence. Each plot shows the performance improvement from eliminating dead-entry re-walks against a different candidate predictor for all 24 workloads. Figure 1(a) shows that dead-entry ratio has no predictive power: all TLB-sensitive workloads cluster near 99% dead-entry ratio, yet their performance improvement ranges from near zero to 72%. Figure 1(b) shows that MPKI is necessary but not sufficient: workloads with large IPC improvements and workloads with near-zero improvements remain indistinguishable at similar MPKI levels. Figure 1(c) reveals the discriminator: burstiness, measuring how many pending translation requests are collectively served by a single page walk; workloads with high burstiness achieve large performance improvements, while those with near-zero burstiness achieve near-zero improvements.

In some workloads, all threads in a warp reference the same memory page, so a single TLB eviction stalls many warps simultaneously waiting for one translation to return. Eliminating that redundant re-walk unblocks all of them at once, producing large performance improvements and high observed burstiness. In other workloads, each thread accesses a distinct page from a working set that exceeds TLB capacity. Evictions are driven by the sheer size of the working set, and no replacement-level mechanism can eliminate them because the TLB simply cannot hold all required translations at once. These two root causes lead to opposite outcomes under any

dead-entry protection mechanism, and we characterize and validate both across all 24 workloads in Section 5.

Building on this characterization, we design *DEPOT* (Dead-Entry PrOTection), a lightweight hardware mechanism that targets the first type of workload. DEPOT uses a small Bloom filter to track recently evicted TLB entries and temporarily protects re-installed entries from being displaced again, preventing the same redundant re-walks from compounding. The mechanism requires approximately 3.5 KB of storage, introduces no prediction logic, and falls back gracefully to standard LRU replacement for workloads where it provides no benefit. DEPOT also composes well with LatPC [28], the state-of-the-art TLB prefetching and compaction mechanism, because the two techniques address fundamentally different sources of misses. LatPC reduces misses through stride prediction and entry compaction; DEPOT prevents recently used pages from being discarded prematurely. Together they outperform either mechanism alone by 2 to 7% on TLB-sensitive workloads at negligible additional hardware cost.

In this paper, we make the following contributions.

- **Eviction-history characterization.** We measure GPU L2 TLB misses by eviction history across 24 workloads, establishing dead-entry re-walks as a prevalent, structurally distinct miss class.
- **Two-class taxonomy.** We identify two root causes among TLB-sensitive workloads, interference-driven misses where multiple warps share the same virtual page, and capacity-driven misses where the working set exceeds TLB reach, and validate the taxonomy using 2 MB huge page experiments.
- **DEPOT mechanism and composition.** We design DEPOT (Dead-Entry PrOTection), a Bloom-filter mechanism (1 KB filter, 3.5 KB total) achieving up to 72% performance improvement on interference-driven workloads and 2 to 7% additive improvement on top of LatPC [28], the state-of-the-art TLB prefetching and compaction mechanism.

2. Background

2.1. GPU Address Translation

Modern GPUs execute code in *warps* of 32 lock-step threads [1, 20, 24], so a single memory instruction can generate up to 32 virtual addresses at once. Coalescing [51] reduces this when threads share a page, but memory-intensive workloads with large or strided access patterns routinely generate one or more unique VPNs per warp instruction, creating translation pressure with no direct CPU analog.

Figure 2 shows the pipeline. Each SM hosts a private, fully-associative 32-entry L1 TLB [56, 58], flushed at kernel boundaries; on the Ampere SM86 microarchitecture [50] studied here, 46 SMs operate independently. An L1 miss allocates an MSHR entry; a second request for the same in-flight VPN merges into it rather than issuing a duplicate walk, and both requestors unblock together on fill. Unmerged L1 misses forward to the shared, chip-wide L2 TLB (1024

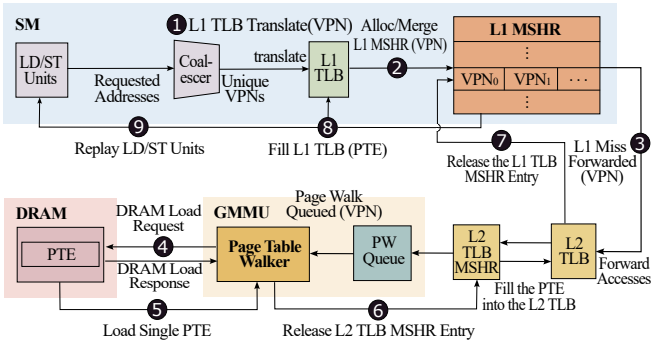

Figure 2: GPU address translation hierarchy (SM86): L1 TLB per SM, shared L2 TLB, 16 PTWs, and MSHR merging for same-VPN requests.

entries on SM86, servicing all 46 SMs), which performs the same same-VPN merge before dispatching to one of 16 hardware page-table walkers (PTWs) [61], each traversing the page table via sequential DRAM reads at hundreds to over a thousand cycles. With 4 KB pages, 1024 L2 entries give 4 MB of reach; footprints beyond that miss regardless of replacement policy. Because only 16 PTWs exist, arriving misses queue behind occupied walkers, and because the L2 MSHR merges by VPN, a single eviction that many warps depend on stalls all of them at once when re-walked – the mechanism behind both the disproportionate cost of a dead-entry miss and the disproportionate recovery from protecting one. This PTW bottleneck is the primary reason TLB behavior dominates performance in translation-sensitive GPU workloads [5, 28, 56].

2.2. Dead-Entry TLB Misses

We define a *dead-entry TLB miss* formally:

> *A TLB miss at virtual page p at time t is a **dead-entry miss** if p was previously installed in the TLB, subsequently evicted before time t, and the eviction preceded the miss without any intervening access to p that would have re-established a valid entry.*

Equivalently, the page-table walker reconstructs a translation that was valid and present within the current execution window, then discarded by LRU replacement [11, 46] – the walk is logically redundant, not new or stale. The classical three-C taxonomy [29] describes *why* an eviction occurred; dead-entry is orthogonal, describing *what* the evicted entry was, and can arise from either a capacity eviction (working set exceeds reach) or an interference-driven eviction (a burst of other pages displaces a still-needed entry) – the two root causes behind the workload classes of Section 5. Not every capacity or conflict eviction produces a dead-entry event: a page never re-accessed within the execution window produces no re-walk. This orthogonality is what makes dead-entry misses actionable where cold and capacity misses are not: a cold miss needs prediction, a capacity miss needs more reach, but a dead-entry miss is avoidable in principle simply

by remembering the recent eviction and protecting the re-installed entry from immediate re-eviction – the observation that motivates DEPOT.

3. Related Work

TLB prefetching and compaction. LatPC [28] pairs a stride predictor [34] with entry compaction to prefetch ahead of demand misses; Valkyrie [10] extends this with multi-level, cross-SM prefetching, Avatar [53] across address-space boundaries, SnakeByte [40] via adaptive recursive page-merging, and STAR [41, 43] via sub-entry sharing on multi-instance GPUs. On CPUs, translation-triggered [12, 13] and learned/pipelined [44, 45] prefetching operate similarly. All predict future translations from access history; we instead characterize misses by eviction history, an orthogonal signal.

Page-walk optimization. R2D2 [27] exploits per-warp address linearity to eliminate redundant walks; Heliostat [21] repurposes ray-tracing hardware for parallel walks; Trans-FW [42] and Barre Chord [22] target multi-GPU/multi-chip-module translation; SpecTLB [7, 8] speculates ahead of confirmed translations and Griffin [9] supports multi-GPU page migration. These reduce the *cost* of walks that occur; we identify a complementary class where the walk itself is redundant.

Huge pages and page-table layout. Mosaic [4] provides range-TLB reach without OS support; CoLT [55] uses contiguous entries at 4 KB granularity; Elastic Cuckoo Page Tables [62] and Translation Ranger [35, 65] restructure layout for parallelism and contiguity-awareness; Pham et al. [52, 54] exploit natural translation clustering. These address the capacity limit directly; we use 2 MB pages only as a diagnostic to separate capacity-driven from interference-driven misses.

TLB-aware scheduling and dead-block prediction. CTA scheduling [39] and MASK [5, 56] improve locality and multi-application concurrency; OASIS [64] places memory object-aware across GPUs; batch-aware management [38, 61] amortizes translation overhead for irregular workloads – these reduce miss *frequency* through scheduling, whereas our structural cause persists regardless. Dead-block prediction has a long cache lineage [31–33]; Khan et al. [37] introduced sampling-based prediction for LLCs, and closest to our work, Mazumdar et al. [47] jointly predict dead pages and blocks via PC-based reuse. We instead use eviction history, requiring no PC tracking, and quantify prevalence across 24 GPU workloads – a structurally distinct miss class not previously isolated at the GPU L2 TLB.

MSHR and CPU TLB characterization. MSHR occupancy is known to carry workload-classification signal for cache/memory tuning [26, 59]; we apply an analogous signal to the TLB MSHR. CPU TLB studies [14, 15, 58] attribute misses mainly to working-set overflow; GPUs differ in that thousands of concurrent warps interact through one shared L2 TLB and MSHR, producing amplification effects absent on CPUs.

4. Methodology

4.1. Simulation Platform

All experiments run on Accel-Sim [36], a cycle-accurate GPU simulator built on GPGPU-Sim [6]. We adopt Accel-Sim's default `SM86_RTX3070` configuration [36], which models an NVIDIA GeForce RTX 3070-class GPU [49] based on the Ampere microarchitecture [2, 50]; the baseline GPU, cache, and memory parameters are used unmodified. The TLB hierarchy parameters—L1 and L2 TLB capacities, MSHR depths, lookup latencies, and the 16 page-table walkers—follow the configuration used in the state-of-the-art work [28], ensuring our results are directly comparable to that prior work. Table 1 summarizes the configuration. The L2 TLB has 1024 entries with LRU replacement, providing 4 MB of reach with 4 KB pages. Page-table walks are modeled with 16 dedicated PTWs, each capable of one concurrent walk. The L2 MSHR supports 128 in-flight translation requests; same-VPN requests are merged into a single MSHR entry.

TABLE 1 Simulated System Configuration.

GPU Configuration	
SM	46 SMs, 1536 threads / 32-thread warps, 32 CTAs per SM 65,536 registers per SM, up to 100 KB shared memory 1132 MHz core clock
Cache	128 B line; L1 D-cache 128 KB, 32-way, 384 MSHRs L2 cache (unified) 4 MB, 16-way
DRAM	GDDR6, 16 channels, 14 Gbps, 3500.5 MHz 254-cycle access latency
Virtual Memory Configuration	
L1 TLB	32 entries, fully associative, 4 KB pages 16 MSHRs (4-way merge), 4 ports 20-cycle lookup latency, LRU replacement
L2 TLB	1024 entries, 16-way, 4 KB pages 128 MSHRs (8-way merge), 16 ports 80-cycle lookup latency, LRU replacement Reach 4 MB (4 KB) / 256 MB (2 MB pages)
Page Walk	16 page-table walkers x86-64 4-level page table [3, 30], 254 cycles per level 32-entry page-walk cache, 20-cycle lookup

4.2. Instrumentation

We extend Accel-Sim with four measurement modules: a *miss arrival timeseries* recording every L2 TLB miss's arrival timestamp; *dead-entry detection*, which tracks the last eviction cycle per VPN in a hash table and classifies a miss as dead-entry if the VPN appears there; an *MSHR dead-slot timeseries*, sampling at 100-cycle intervals the number of MSHR entries awaiting dead-entry re-walks, whose peak defines the *burstiness* metric; and *DEPOT protection statistics* (Bloom filter insert/hit counts, protected-entry counts, LRU fallback events; implementation details in Section 6).

4.3. Workload Suite

Table 2 characterizes all 24 workloads (Classes: **A** = interference-driven; **B** = capacity-driven; **—** = not TLB-sensitive). Here, MPKI is L2 TLB misses per kilo-instruction, while mem-MPKI is misses per kilo memory-instruction, isolating translation pressure from non-memory compute. We select applications with working sets exceeding 4 MB

TABLE 2 List of Evaluated Workloads.

Workload	Suite	MPKI	mem-MPKI	Footprint	Cls
atax	PolyBench	81.4	119.6	16.0 MB	A
mvt	PolyBench	56.5	83.0	16.0 MB	A
bicg	PolyBench	56.4	82.8	16.0 MB	B
gesummv	PolyBench	175.5	249.7	31.3 MB	B
nw	Rodinia	5.02	67.9	32.0 MB	B
dmr	Lonestar	2.99	18.6	24.2 MB	B
kmeans	Rodinia	2.09	72.5	30.3 MB	B
sssp	Lonestar	1.58	11.3	48.0 MB	B
mst	Lonestar	0.98	6.80	73.0 MB	B
backprop	Rodinia	0.02	0.30	9.0 MB	—
bfs	Rodinia	0.03	0.20	2.4 MB	—
gaussian	Rodinia	0.00	0.01	0.5 MB	—
hybridsort	Rodinia	0.02	0.46	12.0 MB	—
lud	Rodinia	0.03	0.58	16.0 MB	—
pagerank	Pannotia	0.06	0.40	10.9 MB	—
p-bfs	Parboil	0.00	0.07	1.1 MB	—
mri-gridding	Parboil	0.09	6.04	226.4 MB	—
sad	Parboil	0.02	1.48	8.5 MB	—
sgemm	Parboil	0.00	0.07	12.0 MB	—
spmv	Parboil	0.19	0.71	29.4 MB	—
2mm	PolyBench	0.00	0.00	5.0 MB	—
fdtd2d	PolyBench	0.09	0.49	48.0 MB	—
gramschmidt	PolyBench	0.40	0.62	24.4 MB	—
streamcluster	Rodinia	0.17	0.70	8.4 MB	—

or L2 TLB MPKI above 1.0, matching the criteria used in LatPC [28]. Workloads span Rodinia [19], PolyBench [25], Lonestar [17], Parboil [63], and Pannotia [18] benchmark suites. We apply a two-bin framing: **TLB-sensitive** workloads (L2 MPKI $\geq$ 1, matching the selection criterion of LatPC [28]) vs. **not TLB-sensitive** workloads (L2 MPKI $<$ 1). Nine workloads meet the TLB-sensitive threshold (including `mst` at MPKI = 0.98, a near-threshold borderline case with confirmed capacity-driven root cause); the remaining 15 serve as a graceful-degradation validation set. Within the TLB-sensitive bin, we sub-classify by access structure: **Class A** (interference-driven) for workloads where all threads in a warp reference the same virtual page (shared VPN)—producing MSHR-merge amplification—and **Class B** (capacity-driven) for workloads where each warp instruction generates distinct VPNs per thread, driving working-set-overflow evictions that exceed TLB reach. Class B membership is confirmed by the 2 MB IPC experiment: if switching to 2 MB pages (L2 reach = 256 MB) yields a large IPC speedup, the workload was reach-limited.

4.4. Evaluated Configurations

We evaluate six configurations: (i) Baseline: Unmodified LRU replacement, 4 KB pages, (ii) DEPOT: DEPOT eviction protection enabled with default parameters (W = 500K cycles, 8192-bit Bloom filter), (iii) 2 MB baseline: Baseline with 2 MB huge pages (L2 TLB reconfigured to 128 entries $\times$ 2 MB, reach = 256 MB), (iv) 2 MB+DEPOT: DEPOT with 2 MB pages, (v) LatPC: State-of-the-art TLB prefetching and compaction [28] enabled, 4 KB pages, and (vi) LatPC+DEPOT: Both LatPC and DEPOT enabled simultaneously.

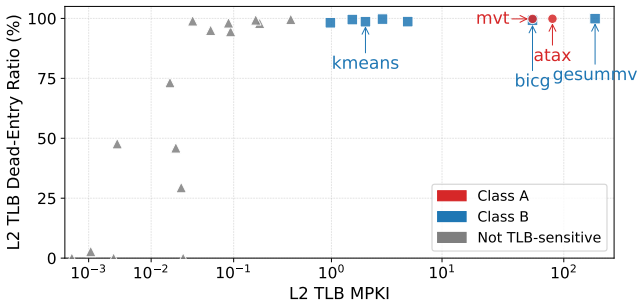

Figure 3: L2 TLB Dead-entry ratio vs. L2 TLB MPKI for all 24 workloads.

5. Dead-Entry Miss Characterization

5.1. Dead-Entry Ratio

Dead-entry misses are not a rare edge case but the dominant miss type across the suite: 16 of 24 workloads exhibit dead-entry ratios above 90%, and all 9 TLB-sensitive workloads exceed 98% (six above 99%). This near-unity ratio arises because a TLB-sensitive workload's working set is large enough to force eviction between successive accesses to the same page, yet small enough that the same pages recur within a kernel – so virtually every steady-state L2 miss re-walks a page the TLB has already translated and discarded. High DE ratio alone does not imply TLB sensitivity, however: `lud` and `gramschmidt` exceed 99% DE ratio yet sustain high throughput because their absolute miss rate is too low for cumulative stall time to matter, motivating a joint condition of high DE ratio *and* high MPKI.

Figure 3 plots all 24 workloads by DE ratio against MPKI. A vertical gap separates the 9 TLB-sensitive workloads, at elevated MPKI, from the other 15, and within that high-MPKI band every workload clusters near 100% DE ratio – confirming near-unity DE ratio as a reliable marker of TLB-sensitive steady state. Below the gap, DE ratio spans near-zero to above 99% with no relation to MPKI, confirming it carries no predictive power on its own.

To connect this characterization to DEPOT's mechanism, we measure the eviction-to-reaccess distance for each dead-entry re-walk. Figure 4 shows the average distance per TLB-sensitive workload against DEPOT's default 500 K-cycle protection window: every workload's average sits well under the window, and across all nine, 83–100% of individual re-walks occur $\geq$1K cycles after eviction, with a modal bin of 10K–99K cycles for six of nine – far exceeding what the 1024-entry L2 TLB can retain under LRU alone. `dmr`, `mst`, and `sssp` (hatched) reflect a baseline run that had not fully converged when measured; direction is consistent with the other six but exact values may shift.

Reuse distance itself, however, does not separate the two classes (`atax` and `bicg` show comparable distributions despite their opposite DEPOT response), reinforcing that access structure, not miss timing, is the discriminator developed next.

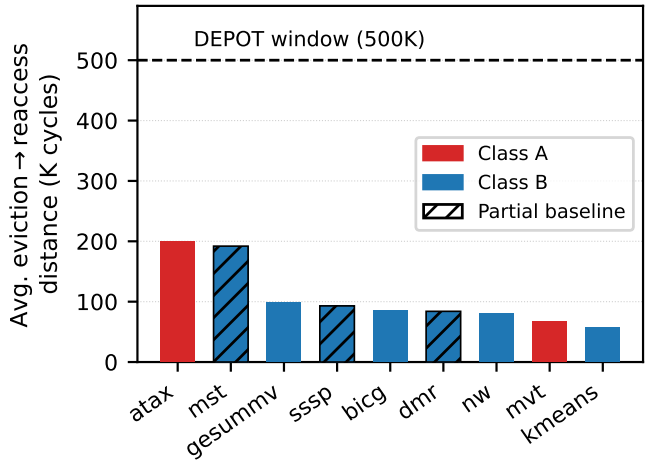

Figure 4: Average eviction-to-reaccess distance per TLB-sensitive workload vs. DEPOT's default protection window. Hatched bars (`dmr`, `mst`, `sssp`) are from a partially-converged baseline.

5.2. Access Structure: The Class Discriminator

`atax` and `bicg` illustrate why miss statistics alone are insufficient: they share nearly identical DE ratios (∼99.9%/∼99.2%) and comparable MPKI (81/56), and even their miss arrival rates track closely through steady state, yet their response to the same eviction-history protection differs by nearly two orders of magnitude. The discriminator is access structure – whether warps reference the same virtual page or distinct pages at a given moment – and Figure 5 makes it visible directly: `atax` shows brief, high-amplitude occupancy bursts, while `bicg` is flat and sustained throughout.

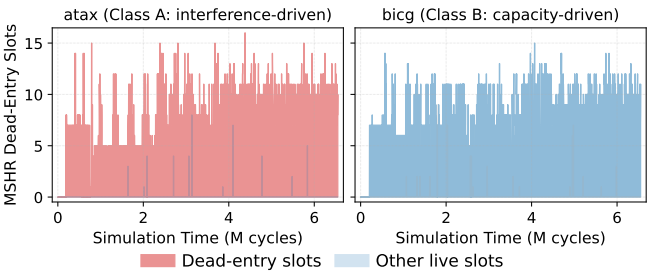

Figure 5: MSHR dead-slot occupancy over time for `atax` and `bicg`.

In `atax`, each warp computes y += $A[i][j] \cdot x[j]$: all 32 threads access the same element of $x[]$, producing one shared VPN per warp instruction. When that VPN is evicted and re-walked, every warp stalled on it merges into a single MSHR entry, so one page walk releases many warps at once, producing the sharp spikes in Figure 5 (spike height tracks merge depth); `mvt` shares this pattern for analogous algebraic reasons, forming Class A's second member. In `bicg`, each warp computes a column dot product where thread i accesses row i of A, producing 32 distinct VPNs per warp instruction with a merge depth near one – its 2048-by-2048 working set needs ∼4096 pages, four times L2 TLB capacity, so evictions are capacity-driven and the occupancy trace stays flat, a structure shared by the remaining six Class B workloads (`nw`, `sssp`, `kmeans`, `gesummv`, `dmr`, `mst`). The two classes are thus distinguished by occupancy shape alone – bursty spikes mean shared pages and disproportionate per-eviction recovery, flat sustained occupancy means capacity-bound evictions no replacement policy can resolve – observable from existing MSHR counters with no offline profiling required.

5.3. Validating Capacity-Driven Misses

If the seven Class B workloads are genuinely capacity-driven, expanding TLB reach should eliminate their dead-entry misses. Figure 6 shows IPC speedup under 2 MB huge pages (L2 reach 4 MB → 256 MB) over the 4 KB baseline for all nine TLB-sensitive workloads.

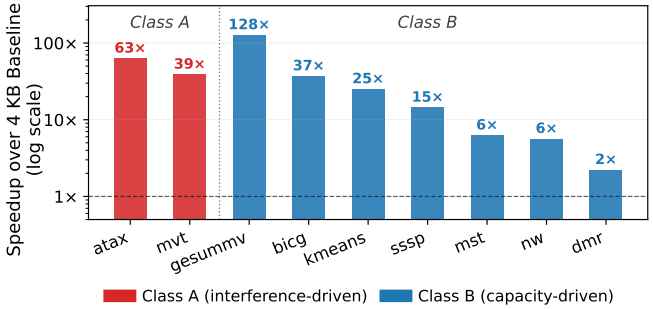

Figure 6: IPC speedup under 2 MB pages over 4 KB baseline for all 9 TLB-sensitive workloads.

Results confirm the diagnosis for all 7 Class B workloads, with speedups tracking working-set size relative to the expanded reach: `gesummv` 128×, (16 MB set fits entirely), `bicg` 37×, `kmeans` 25×, `sssp` 14.5×, `mst` 6.3×, `nw` 5.7×, and `dmr` 2.2× (a more irregular address stream, only partially contained). Class A also benefits substantially (`atax` 63×, `mvt` 39×), but unlike Class B, its performance is not fundamentally capacity-constrained – expanding reach does not eliminate the interference among warps competing for the same entries.

6. DEPOT: Dead-Entry PrOTection

In this section, we propose DEPOT that targets Class A dead-entry re-walks through a three-phase mechanism detection, fill, and eviction implemented entirely within the L2 TLB pipeline. Figure 7 shows the hardware. Three additions extend the existing L2 TLB datapath: an eviction-history Bloom filter, a small Pending Dead-Entry Set (PDS) that bridges the miss and fill events, and a Protection Timestamp Writer that drives a new 20-bit per-entry protection-timer field in the tag array.

Detection (miss path). On every L2 TLB miss, the Dead-Entry Detection Logic receives the missed VPN ❶ and hashes it through three independent hash functions (h_1, h_2, h_3) into an 8192-bit Bloom filter [16] that records recently evicted

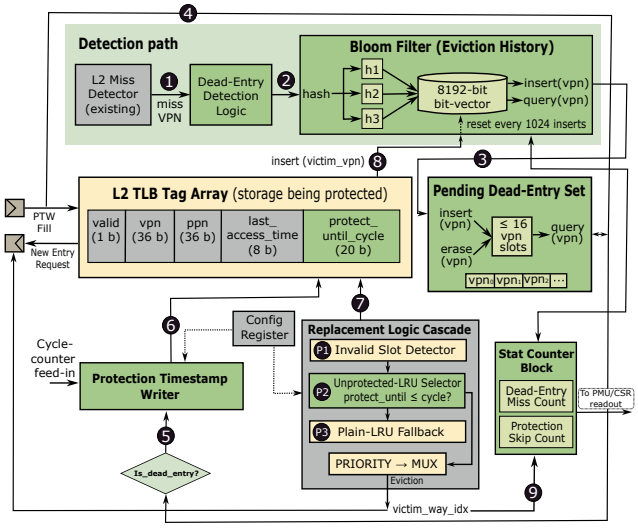

Figure 7: DEPOT mechanism: eviction-history Bloom filter, Pending Dead-Entry Set, per-entry protection timer, and protection-aware replacement cascade.

VPNs ❷. A filter hit indicates that the missed VPN was recently in the TLB and is therefore a probable dead-entry re-walk, so the in-flight miss is registered in the PDS ❸- a small ($\leq$ 16 slot) buffer that holds VPNs awaiting fill. Misses whose VPN is absent from the filter (genuine cold or capacity misses with no eviction history) bypass the PDS and proceed through the standard page-walk path.

Fill (protection path). When the page-table walk completes, the fill triggers a PDS query for the filling VPN ❹. A PDS hit activates the Protection Timestamp Writer ❺, which reads the cycle counter and sets the newly installed entry's protection timer to expire at $now + W$ cycles ❻, where W defaults to 500 K cycles and is held in a configuration register. The PDS slot for that VPN is then released. Fills whose VPN was not previously flagged install with the protection field untouched and behave identically to baseline.

Eviction (protection-aware replacement). On every fill that requires a victim, the Replacement Logic Cascade reads each entry's protection timer from the tag array ❼ and selects through three priority stages: *P1 Invalid Slot Detector* returns any invalid entry immediately; *P2 Unprotected-LRU Selector* compares each entry's protection timer against the current cycle and selects the LRU entry among those whose protection has expired; *P3 Plain-LRU Fallback* fires only when every candidate in the set is still protected, in which case the cascade reverts to standard LRU. The selected victim's VPN is inserted into the Bloom filter, refreshing the eviction history ❽, and the victim's way index is forwarded to a small Stat Counter Block ❾ that records dead-entry-miss and protection-skip counts. This graceful degradation bounds DEPOT's worst-case overhead to zero, since the fallback path is indistinguishable from baseline LRU. Two details keep the mechanism stable long-run: the filter resets every 1024

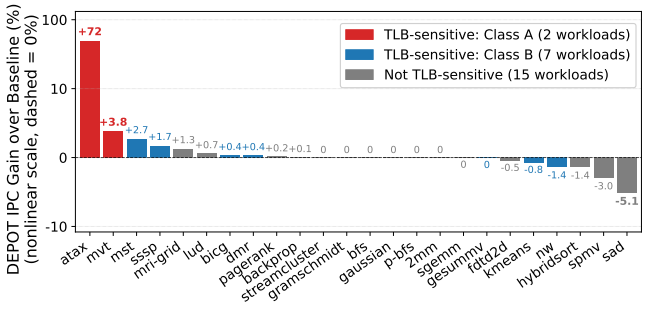

Figure 8: IPC improvement of DEPOT over 4 KB baseline for all 24 workloads, sorted descending.

insertions (a count-based threshold keeping expected false-positive rate below 5% while avoiding the temporal aliasing a cycle-timer reset could cause across kernel boundaries), and protection state flushes at each kernel boundary so a VPN evicted in kernel K_n cannot spuriously protect an entry in K_{n+1}, where the same virtual address may map to a different physical frame after relaunch.

Hardware cost is 1 KB for the Bloom filter plus 1024 $\times$ 20 bits = 2.5 KB for the per-entry protection timers across the 1024-entry L2 TLB, totaling 3.5 KB amortized across the existing array. The design is structurally similar to the evicted-address filter of Seshadri et al. [60] for cache pollution/thrashing mitigation, applied here to drive protection rather than insertion policy.

7. Performance Evaluation and Analysis

7.1. DEPOT Performance

Figure 8 shows IPC improvement of DEPOT for all 24 workloads. The results divide cleanly along the class boundaries established in Section 5. Among the 15 not-TLB-sensitive workloads, IPC improvement is near zero throughout, with occasional small negative values including `sad` at –5.1% and `spmv` at –2.9%, from second-order LRU displacement in partially protected sets rather than translation overhead – both have MPKI below 0.2, so TLB stalls are negligible at baseline. Among the two Class A workloads, `atax` improves from 0.354 to 0.610 IPC (+72%), reflecting near-complete elimination of dead-entry re-walk stalls, and `mvt` improves from 0.574 to 0.596 (+3.8%); the smaller gain reflects `mvt`'s lower peak MSHR merge depth (15 vs. `atax`'s 17). All seven Class B workloads show near-zero DEPOT gain (–1.4% to +2.7%, within simulation noise) across PolyBench, Lonestar, and Rodinia workloads alike: when each warp generates distinct VPNs, protecting one re-installed entry is immediately offset by another capacity eviction, while the LRU fallback ensures zero overhead.

Figure 9 plots IPC improvement against MPKI, revealing a clean two-regime structure: workloads below MPKI = 1 cluster near zero gain regardless of dead-entry ratio (miss rate is necessary but not sufficient), while within the high-MPKI group, access structure – not MPKI – determines whether protection helps. A worst-case fully-saturated Bloom filter

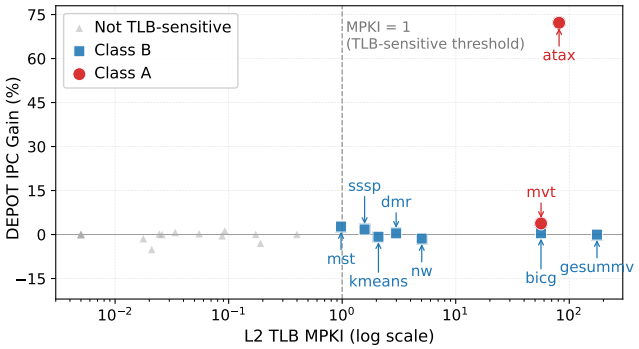

Figure 9: IPC improvement of DEPOT vs. L2 TLB MPKI for all 24 workloads.

(every query a false positive) leaves `atax` IPC unchanged at 0.610, since the LRU fallback absorbs the mismatch without measurable cost. Sweeping DEPOT's two parameters, the protection window (100 K–2 M cycles) and Bloom filter size (2048–16384 bits), across the 9 TLB-sensitive workloads shows both nearly flat across the full tested range: re-walk bursts fall within any reasonable window, and the eviction stream stays well below saturation for even the smallest filter. The default (500 K cycles, 8192 bits) is effective without tuning.

7.2. Class A/B Asymmetry

The asymmetry follows directly from the access-structure analysis of Section 5.2. In Class A, a single dead-entry eviction is amplified through MSHR merging because many warps reference the same page; protecting the re-installed entry eliminates the initial re-walk and every merged re-walk that would otherwise stall those warps simultaneously. At SM86's ∼300–500-cycle page-walk latency, releasing dozens of stalled warps in one protection event recovers thousands of cycles, consistent with the +72% gain. In Class B, the TLB is continuously occupied by a working set exceeding its reach; protecting one entry only shifts the next eviction elsewhere, MSHR merging is minimal so each walk serves one warp, and the LRU fallback (triggered when all entries in a set are protected) keeps overhead negligible.

7.3. Composition with LatPC

LatPC combines stride-based prefetching with entry compaction, targeting cold and capacity-driven misses by predicting future accesses ahead of demand – a different point in the TLB pipeline than DEPOT's replacement-boundary protection, so the two can compose additively or contend depending on workload.

LatPC alone achieves large gains on TLB-sensitive workloads (`atax` +1227% to 4.70 IPC, `bicg` +667% to 4.61, `mvt` +729% to 4.76) by eliminating the dominant cold-miss bottleneck. LatPC+DEPOT improves Class A further (`atax` +5.4% to 4.95, `mvt` +4.0% to 4.95) because LatPC's forward-looking predictor cannot catch an entry displaced by LRU *after* a prefetch installs a new one but *before* the next stride access arrives – exactly the residual class DEPOT's replacement-boundary protection covers, at the marginal cost of a 1 KB filter. Class B results are mixed: `bicg` (+6.9%), `mst` (+2.5%), and `sssp` (+2.1%) gain modestly and `dmr` is flat, but `kmeans` (–8.3%), `nw` (–10.0%), and `gesummv` (–28.5%) regress, because DEPOT's protected entries block replacement slots that LatPC's prefetch fills need in capacity-saturated workloads. This is consistent with the taxonomy rather than a counterexample: the regressions occur specifically when an interference-driven mechanism is misapplied atop a prefetcher on capacity-driven workloads, not randomly.

8. Discussion and Limitations

DEPOT and 2 MB pages (Section 5.3) address different root causes and are not interchangeable: larger pages relieve Class B capacity eviction but not Class A interference, and are not always available (custom allocators, virtualized guests without huge-page support), whereas DEPOT is transparent to OS, driver, and application. DEPOT targets interference-driven misses specifically and provides no benefit – though also no harm in isolation – on capacity-driven workloads; practitioners should use the two-class framework to identify root cause before applying it, since composing it with an aggressive prefetcher like LatPC on a capacity-driven workload can regress performance by blocking the prefetcher's replacement slots (Section 7.3). A capacity-aware coupling – e.g., suspending protection while the prefetcher is actively installing – is a natural direction for future work. Results are calibrated to the SM86 Ampere configuration studied here; the qualitative MSHR-merge-amplification mechanism should generalize to GPUs sharing this L1/L2 TLB organization, but absolute thresholds and gains may shift on architectures with substantially different TLB capacity or walker count.

9. Conclusion

Dead-entry TLB misses are a structurally distinct, avoidable miss class that accounts for over 99% of L2 TLB misses in the most TLB-sensitive GPU workloads. We characterize dead-entry misses across 24 GPU workloads, using MPKI ≥ 1 as a TLB-sensitivity threshold to identify 9 TLB-sensitive workloads and sub-classify them by access structure into Class A and Class B, with Class B membership validated by 2 MB IPC experiments confirming reach-limited root causes. DEPOT, a 1 KB Bloom-filter eviction protection mechanism, eliminates Class A dead-entry re-walks with up to 72% IPC improvement, degrades gracefully to zero overhead on Class B, and composes additively with LatPC [28] on interference-driven workloads to deliver 2 to 7% further improvement at negligible marginal hardware cost, pushing `atax` from 4.70 to 4.95 IPC for a combined 14× improvement over the 4 KB baseline. Several directions remain for future work. The temporal MSHR occupancy signatures identified in this paper suggest the potential for runtime-adaptive TLB policies that dynamically distinguish

interference-driven from capacity-driven miss behavior. In addition, integrating eviction-history signals with TLB-aware warp scheduling or page-placement mechanisms may further reduce destructive translation interference before evictions occur.

References

[1] T. M. Aamodt, W. W. L. Fung, and T. G. Rogers, *General-Purpose Graphics Processor Architectures*, ser. Synthesis Lectures on Computer Architecture. Morgan & Claypool, 2018.

[2] H. Abdelkhalik, Y. Arafa, N. Santhi, and A.-H. Badawy, "Demystifying the Nvidia Ampere architecture through microbenchmarking and instruction-level analysis," arXiv:2208.11174, 2022.

[3] Advanced Micro Devices, Inc., "AMD64 architecture programmer's manual," 2024.

[4] R. Ausavarungnirun, J. Landgraf, V. Miller, S. Ghose, J. Gandhi, C. J. Rossbach, and O. Mutlu, "Mosaic: A GPU memory manager with application-transparent support for multiple page sizes," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2017.

[5] R. Ausavarungnirun, V. Miller, J. Landgraf, S. Ghose, J. Gandhi, A. Jog, C. J. Rossbach, and O. Mutlu, "MASK: Redesigning the GPU memory hierarchy to support multi-application concurrency," in *Proc. International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2018.

[6] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, "Analyzing CUDA workloads using a detailed GPU simulator," in *Proc. International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2009.

[7] T. W. Barr, A. L. Cox, and S. Rixner, "Translation caching: Skip, don't walk (the page table)," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2010.

[8] T. W. Barr, A. L. Cox, and S. Rixner, "SpecTLB: A mechanism for speculative address translation," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2011.

[9] T. Baruah, Y. Sun, A. T. Dincer, S. A. Mojumder, J. L. Abellán, Y. Ukidave, A. Joshi, N. Rubin, J. Kim, and D. Kaeli, "Griffin: Hardware-software support for efficient page migration in multi-GPU systems," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2020.

[10] T. Baruah, Y. Sun, S. A. Mojumder, J. L. Abellán, Y. Ukidave, A. Joshi, N. Rubin, J. Kim, and D. Kaeli, "Valkyrie: Leveraging inter-TLB locality to enhance GPU performance," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2020.

[11] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[12] A. Bhattacharjee, "Large-reach memory management unit caches," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2013.

[13] A. Bhattacharjee, "Translation-triggered prefetching," in *Proc. International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017.

[14] A. Bhattacharjee, D. Lustig, and M. Martonosi, "Shared last-level TLBs for chip multiprocessors," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2011.

[15] A. Bhattacharjee and M. Martonosi, "Inter-core cooperative TLB prefetchers for chip multiprocessors," in *Proc. International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2010.

[16] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.

[17] M. Burtscher, R. Nasre, and K. Pingali, "A quantitative study of irregular programs on GPUs," in *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, 2012.

[18] S. Che, B. M. Beckmann, S. K. Reinhardt, and K. Skadron, "Pannotia: Understanding irregular GPGPU graph applications," in *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, 2013.

[19] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A benchmark suite for heterogeneous computing," in *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, 2009.

[20] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, and K. Skadron, "A performance study of general-purpose applications on graphics processors using CUDA," *Journal of Parallel and Distributed Computing*, vol. 68, no. 10, 2008.

[21] Y. Feng, Y. Li, J. Lee, W. W. Ro, and H. Jeon, "Heliostat: Harnessing ray tracing accelerators for page table walks," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2024.

[22] Y. Feng, S. Na, H. Kim, and H. Jeon, "Barre chord: Efficient virtual memory translation for multi-chip-module GPUs," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2024.

[23] D. Ganguly, Z. Zhang, J. Yang, and R. Melhem, "Interplay between hardware prefetcher and page eviction policy in CPU-GPU unified virtual memory," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2019.

[24] M. Garland, S. Le Grand, J. Nickolls, J. Anderson, J. Hardwick, S. Morton, E. Phillips, Y. Zhang, and V. Volkov, "Parallel computing experiences with CUDA," *IEEE Micro*, vol. 28, no. 4, 2008.

[25] S. Grauer-Gray, L. Xu, R. Searles, S. Ayalasomayajula, and J. Cavazos, "Auto-tuning a high-level language targeted to GPU codes," in *Proc. Innovative Parallel Computing (InPar)*, 2012.

[26] Y. Gu and L. Chen, "Dynamically linked MSHRs for adaptive miss handling in GPUs," in *Proc. International Conference on Supercomputing (ICS)*, 2019.

[27] D. Ha, Y. Oh, and W. W. Ro, "R2D2: Removing redundancy utilizing linearity of address generation in GPUs," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2023.

[28] Y. Ha *et al.*, "LATPC: Locality-aware TLB prefetching and MSHR compression," in *Proc. International Symposium on Microarchitecture (MICRO)*, Seoul, Republic of Korea, Oct. 2025.

[29] M. D. Hill and A. J. Smith, "Evaluating associativity in CPU caches," *IEEE Transactions on Computers*, vol. 38, no. 12, pp. 1612–1630, 1989.

[30] Intel Corporation, "Intel® 64 and IA-32 architectures software developer's manual," 2024.

[31] A. Jain and C. Lin, "Back to the future: Leveraging Belady's algorithm for improved cache replacement," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2016.

[32] A. Jaleel, K. B. Theobald, S. C. Steely, and J. Emer, "High performance cache replacement using re-reference interval prediction (RRIP)," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2010.

[33] N. P. Jouppi, "Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers," in *Proc. International Symposium on Computer Architecture (ISCA)*, 1990.

[34] G. B. Kandiraju and A. Sivasubramaniam, "Going the distance for TLB prefetching: An application-driven study," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2002.

[35] V. Karakostas, J. Gandhi, F. Ayar, A. Cristal, M. D. Hill, K. S. McKinley, M. Nemirovsky, M. M. Swift, and O. Ünsal, "Redundant memory mappings for fast access to large memories," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2015.

[36] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-Sim: An extensible simulation framework for validated GPU modeling," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2020.

[37] S. M. Khan, Y. Tian, and D. A. Jiménez, "Sampling dead block prediction for last-level caches," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2010.

[38] H. Kim, J. Sim, P. Gera, R. Hadidi, and H. Kim, "Batch-aware unified memory management in GPUs for irregular workloads," in *Proc. International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.

[39] G. Koo, H. Jeon, Z. Liu, N. S. Sung, and M. Annavaram, "CTA-aware prefetching and scheduling for GPU," in *Proc. IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2018.

[40] J. Lee, J. M. Lee, Y. Oh, W. J. Song, and W. W. Ro, "SnakeByte: A TLB design with adaptive and recursive page merging in GPUs," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2023.

[41] B. Li, Y. Wang, T. Wang, L. Eeckhout, J. Yang, and X. Tang, "STAR: Sub-entry sharing-aware TLB for multi-instance GPU," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2024.

[42] B. Li, J. Yin, A. Holey, Y. Zhang, J. Yang, and X. Tang, "Trans-FW: Short circuiting page table walk in multi-GPU systems via remote forwarding," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2023.

[43] B. Li, J. Yin, Y. Zhang, and X. Tang, "Improving address translation in multi-GPUs via sharing and spilling aware TLB design," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2021.

[44] A. Margaritov, D. Ustiugov, E. Bugnion, and B. Grot, "Virtual address translation via learned page table indexes," in *Workshop on ML for Systems at NeurIPS*, 2018.

[45] A. Margaritov, D. Ustiugov, E. Bugnion, and B. Grot, "Prefetched address translation," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2019.

[46] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger, "Evaluation techniques for storage hierarchies," *IBM Systems Journal*, vol. 9, no. 2, pp. 78–117, 1970.

[47] C. Mazumdar, P. Mitra, and A. Basu, "Dead page and dead block predictors: Cleaning TLBs and caches together," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2021.

[48] S. Mostofi, H. Falahati, N. Mahani, P. Lotfi-Kamran, and H. Sarbazi-Azad, "Snake: A variable-length chain-based prefetching for GPUs," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2023.

[49] NVIDIA Corporation, "NVIDIA GeForce RTX 3070 family," https://www.nvidia.com/en-us/geforce/graphics-cards/30-series/rtx-3070-3070ti/, 2020.

[50] NVIDIA Corporation, "NVIDIA ampere GA102 GPU architecture whitepaper," https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf, 2021.

[51] L. Nyland, J. R. Nickolls, G. Hirota, and T. Mandal, "Systems and methods for coalescing memory accesses of parallel threads," US Patent No. 8,086,806, 2011.

[52] C. H. Park, T. Heo, J. Jeong, and J. Huh, "Hybrid TLB coalescing: Improving TLB translation coverage under diverse fragmented memory allocations," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2017.

[53] J. Park, Y. L. Kwon, S. Jeong, G. B. Hong, J. Yoon, P. J. Nair, and S. Hong, "A case for speculative address translation with rapid validation for GPUs," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2024.

[54] B. Pham, A. Bhattacharjee, Y. Eckert, and G. H. Loh, "Increasing TLB reach by exploiting clustering in page translations," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2014.

[55] B. Pham, V. Vaidyanathan, A. Jaleel, and A. Bhattacharjee, "CoLT: Coalesced large-reach TLBs," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2012.

[56] B. Pichai, L. Hsu, and A. Bhattacharjee, "Architectural support for address translation on GPUs: Designing memory management units for CPU/GPUs with unified address spaces," in *Proc. International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014.

[57] J. Power, M. D. Hill, and D. A. Wood, "Supporting x86-64 address translation for 100s of GPU lanes," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2014.

[58] J. Power, M. D. Hill, and D. A. Wood, "Supporting x86-64 address translation for 100s of GPU lanes," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2014.

[59] M. K. Qureshi, D. N. Lynch, O. Mutlu, and Y. N. Patt, "A case for MLP-aware cache replacement," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2006.

[60] V. Seshadri, O. Mutlu, M. A. Kozuch, and T. C. Mowry, "The evicted-address filter: A unified mechanism to address both cache pollution and thrashing," in *Proc. International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2012.

[61] S. Shin, G. Cox, M. Oskin, G. H. Loh, Y. Solihin, A. Bhattacharjee, and A. Basu, "Scheduling page table walks for irregular GPU applications," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2018.

[62] D. Skarlatos, A. Kokolis, T. Xu, and J. Torrellas, "Elastic cuckoo page tables: Rethinking virtual memory translation for parallelism," in *Proc. International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.

[63] J. A. Stratton, C. Rodrigues, I.-J. Sung, N. Obeid, L.-W. Chang, N. Anssari, G. D. Liu, and W.-m. W. Hwu, "Parboil: A revised benchmark suite for scientific and commercial throughput computing," University of Illinois at Urbana-Champaign, Tech. Rep. IMPACT-12-01, 2012.

[64] Y. Wang, B. Li, M. T. Ibn Ziad, A. Jaleel, J. Yang, and X. Tang, "OASIS: Object-aware page management for multi-GPU systems," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2025.

[65] Z. Yan, D. Nellans, D. Lustig, and A. Bhattacharjee, "Translation ranger: Operating system support for contiguity-aware TLBs," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2024.